BioNix_Wesley_Week_10_ILP

Tuesday, 23 September 2025 19:20

VirusBreakend Reproducible Packaging

1. Project Goal

This week, my primary objective was to create a fully reproducible packaging solution for VirusBreakend, the viral integration detection tool provided as part of the GRIDSS software suite, as a component of the BioNix internship.

2. Why Not Use Previous "Full Source" Packaging?

Initially, I considered building a native Nix derivation from the official GRIDSS full release asset (ZIP/tarball) as a way to wrap and reproduce VirusBreakend.
However, there were two key blockers:

- 1) The official GRIDSS binary releases (ZIP packages) have been removed from the upstream GitHub repository — currently, only source code tarballs are available, with no easy “download and run” option for the prebuilt toolchain. This means it is impossible to fetch or pin a fixed, reproducible binary version for packaging.

- I tried :

```
gridssVersion = "2.13.2";
gridssSrc = pkgs.fetchurl {
  url = "https://github.com/PapenfussLab/gridss/releases/download/v
${gridssVersion}/gridss-${gridssVersion}.zip";
  sha256 = pkgs.lib.fakeSha256;
};
```

```
error: Cannot build '/nix/store/q3n1ikkh48xwfp70qq2l44prjk9xgrmr-gridss-2.13.2.zip.drv'.
       Reason: builder failed with exit code 1.
       Output paths:
         /nix/store/ndwkmxk1nm5911zn1wsy3hjr8f8y8735-gridss-2.13.2.zip
       Last 7 log lines:
       >
       > trying https://github.com/PapenfussLab/gridss/releases/download/v2.13.2/gridss-2.13.2.zip
       >   % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
       >                                  Dload  Upload   Total   Spent    Left  Speed
       > 0     9    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
       > curl: (56) The requested URL returned error: 404
       > error: cannot download gridss-2.13.2.zip from any mirror
       For full logs, run:
         nix log /nix/store/q3n1ikkh48xwfp70qq2l44prjk9xgrmr-gridss-2.13.2.zip.drv
```

- 2) The GRIDSS software suite is very large and complex, including not only the VirusBreakend script but also large JAR files and numerous runtime dependencies (Java, BWA, samtools, kraken2, etc.). Attempting to rebuild all dependencies and wire everything up from scratch within Nix is non-trivial and not realistic within a single week, especially given the need for strict reproducibility.

```
error: syntax error, unexpected invalid token, expecting '}'
       at /Users/wesley/virusbreakend-nix/flake.nix:38:67:
           37|             if [ -f "$out/share/gridss/$s" ]; then
           38|               install -m 0755 "$out/share/gridss/$s" "$out/bin/${s%.sh}"
             |                                                                   ^
           39|               wrapProgram "$out/bin/${s%.sh}" \
```

```
error: Cannot build '/nix/store/...-gridss-2.13.2.zip.drv'
```

```
error: Cannot build '/nix/store/...-gridss-2.13.2.zip.drv'.
Reason: builder failed with exit code 1.
...
curl: (56) The requested URL returned error: 404
error: cannot download gridss-2.13.2.zip from any mirror
```

As a result, I determined that the best reproducibility strategy is to use the official gridss/gridss Docker image, which is updated and maintained by the developers, and always contains both GRIDSS and VirusBreakend in a fully configured environment.

3. How I Achieved Reproducibility

- I wrote a minimal Nix flake (flake.nix) that wraps the official Docker image and exposes the virusbreakendcommand as a fully reproducible CLI workflow.

```
runner = pkgs.writeShellScriptBin "virusbreakend-docker" ''
 set -euo pipefail
 if ! command -v docker >/dev/null; then
   echo "docker not found. Please install/start Docker." >&2
   exit 1
 fi
 docker run --rm -it \
   -v "$PWD:/work" -w /work \
   gridss/gridss:latest \
   virusbreakend "$@"
'';
```

- This approach ensures any user (with Nix and Docker) can run the exact same VirusBreakend pipeline, regardless of their local environment.

```
apps.virusbreakend = { type = "app"; program = "${runner}/bin/virusbreakend-docker"; };
```

- The flake was tested and validated by running nix run .#virusbreakend -- --help and observing the expected CLI help output, confirming that the workflow is fully functional and reproducible.

```
nix run .#virusbreakend -- --help
nix run .#virusbreakend -- -o sample.vcf -r ref.fa normal.bam tumor.bam
```

4. What Was Successfully Delivered

- A working flake.nix that provides a reproducible VirusBreakend command, fully compatible with the official GRIDSS Docker image.
- A clear README.md for users to follow, describing how to use and extend the workflow for their own data.
- All code tracked and version-controlled in Git for transparent reproducibility.

```
git init
git add flake.nix README.md
git commit -m "init reproducible VirusBreakend flake"
```

```
nix run .#virusbreakend -- --help
```

```
VIRUSBreakend: Viral Integration Recognition Using Single Breakends
Usage: virusbreakend [options] input.bam
```

...

5. Reflection & Next Steps

- This approach maximizes maintainability and reliability by deferring to the official Docker environment provided by the GRIDSS developers, sidestepping issues around package version drift, upstream asset removal, and dependency management.
- For future work, the flake could be extended to expose additional GRIDSS commands, batch pipeline scripts, or even support alternative viral detection workflows, as needed.

Summary sentence for supervisor / diary:

In Week 10, I achieved a fully reproducible VirusBreakend packaging using Nix and the official Docker image, after identifying that full native packaging is not feasible due to removal of binary releases and the complexity of the GRIDSS toolchain.

Q:

1. Why do upstream authors sometimes remove binary releases?
2. How should a reproducibility-focused group handle upstream asset removal?

Wesley

- Working on the VirusBreakend Reproducible Packaging
- Question
    1. Why do upstream authors sometimes remove binary releases?
    2. How should a reproducibility-focused group handle upstream asset removal?